

Automatic Generation of Test Cases and Test Purposes from Natural Language

Sidney Nogueira^{1,2(✉)}, Hugo L.S. Araujo¹, Renata B.S. Araujo¹,
Juliano Iyoda¹, and Augusto Sampaio¹

¹ Centro de Informática, Universidade Federal de Pernambuco, Recife, Brazil

² DEINFO, Universidade Federal Rural de Pernambuco, Recife, Brazil
`sidney.nogueira@ufrpe.br`

Abstract. Use cases are widely used for requirements description in the software engineering practice. As a use case event flow is often written in natural language, it lacks tools for automatic analysis or processing. In this paper, we extend previous work that proposes an automatic strategy for generating test cases from use cases written in a Controlled Natural Language (CNL), which is a subset of English that can be processed and translated into a formal representation. Here we propose a state-based CNL for describing use cases. We translate state-based use case descriptions into CSP processes from which test cases can be automatically generated. In addition, we show how a similar notation can be used to specify test selection via the definition of state-based test purposes, which are also translated into CSP processes. Test generation and selection are mechanised by running refinement checking verifications using the CSP processes for use cases and test purposes. All the steps of the strategy are integrated into a tool that provides a GUI for authoring use cases and test purposes described in the proposed CNL, so the formal CSP notation is totally hidden from the test designer. We illustrate our tool and techniques with a running example.

1 Introduction

Model-based testing (MBT) uses a model to describe the system under test in order to generate test cases automatically. There are many different notations used to create test models. For instance, Finite State Machines (FSM) and Labelled Transition Systems (LTS) are commonly adopted as input for test generation tools [12, 25]. Therefore, the adoption of MBT normally requires the users to be familiar with a formal specification language.

Unfortunately, in some contexts, working with modelling formalisms might be an impediment for the effective use of MBT. Thus, a more light-weight notation supported by an interactive tool might help to reduce the gap between informal specifications adopted in the initial phases of software development processes and a formal representation required by MBT.

Use cases are widely employed in the software engineering practice. It describes interactions between the user and the system in terms of event flows,

typically written in natural language. Use cases are easier to learn in comparison with the formal notations used in MBT. However, as natural language is hard to process automatically, use cases lack tool support for automatic analysis and processing. Some efforts in this direction generate test cases from requirements written in natural language [3, 4, 19, 21]. In [3, 19] the authors do not impose any grammar restrictions on the language. This tends to demand more user intervention to generate test cases, unless there is a substantial knowledge base for a specific domain. The works reported in [4, 21] define standardised notations and provide automated strategies to generate test cases.

In previous work [15] we devised a strategy for the automatic generation of test cases from use cases written in a Controlled Natural Language (CNL), which is a subset of English that can be processed and translated into a formal language, and particularly CSP [11]. The main limitation of this strategy, however, is that states, inputs and outputs are expressed directly using (a variation of) the CSP notation, which, in general, is not accessible for test designers.

Here we propose a state-based CNL that embodies state and state operations, as well as inputs and outputs. Despite being a natural language, it requires from the use case designer familiarity with notions of variables, inputs and outputs. We translate state-based CNL use cases into a CSP representation that includes a memory model to record state information. From such a model, test cases and test data are automatically generated. A similar notation has proved convenient to express test purposes as a mechanism for test selection; this uniform use of our state-based CNL is a distinguishing feature of our approach with respect to related work. Furthermore, test generation is normally achieved by designing and running explicit algorithms that traverse a test model. By using CSP to represent both control behaviour and state information, we are able to generate test cases and test data as counterexamples of refinement verifications using a tool such as FDR [10]. Selection can be based both on the occurrence of events as well as on the system state. Our overall approach is illustrated by a case study, presented as a running example.

The next section gives an overview of CSP and of our previous strategy to test case generation. Section 3 introduces the proposed state-based controlled natural language for use case. The following section addresses the translation of state-based CNL descriptions into CSP. Then, Sect. 5 introduces the CNL notation for test purpose specification. In Sect. 6 we present tool support. In the final section we summarise our results and discuss related and future work.

2 Test Case Generation from CSP Models

This section overviews our previous strategy [15] for test case generation from use cases written in natural language, as well as the CSP notation. Figure 1 gives an overview of the TaRGeT Tool [9] that mechanises the strategy. The main inputs are use case documents that specify the software features to be tested, and test purposes written in CSP that define subsets of test scenarios to be generated. Feature descriptions are then translated to CSP test models that

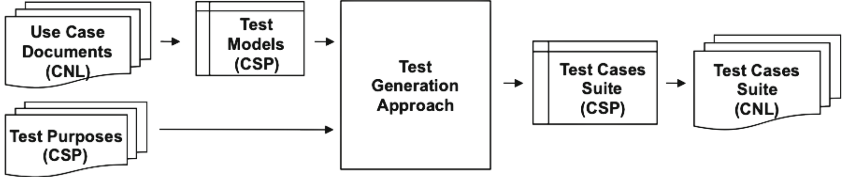


Fig. 1. Test generation approach in the TaRGt tool

capture use cases behaviour. Test models and test purposes are inputs for our test generation approach, which uses FDR [10] to run verifications that yield the desired set of abstract test cases represented as CSP traces (sequences of events). Finally, CNL test cases are obtained from the abstract CSP test suite.

Use Case Documents. Use case documents follow a template that allows the specification of several features; each feature has one or more use cases. Sentences are used to describe user actions and the respective system responses. The CNL is restricted to control aspects. Input, output, guards and variable update can be specified, but they are described in a formal notation (a variant of CSP).

CSP Test Models. For the purpose of test case generation, the use case documents are translated into a formal representation in the CSP process algebra [17, 18]. The main element in CSP is a process, which models a component or an entire system. Processes communicate (atomic and instantaneous) events that represent visible actions. The alphabet α_P denotes the set of events a CSP process P can communicate. Events are communicated as soon as there is an agreement between processes and the environment. *STOP* and *SKIP* are two primitive CSP process. The former represents a deadlocked process that does not communicate any events. The latter represents a process that terminates successfully (communicates the $\checkmark$ event) and deadlocks.

Basic CSP operators as prefix, external choice, and sequential composition are suitable to model the control flow of use cases. The CSP prefix operator $P = ev \rightarrow Q$ specifies that event ev is communicated by P , which then behaves as the process Q . A channel is an important abstraction in CSP for a set of events with a common prefix. Let T be a type and c a channel that can communicate a value of type T (declared as $c : T$). Such a channel declaration represents the set of events $\{c.x \mid x \in T\}$. The external choice operator $P = Q \square R$ indicates that the process P can behave as Q or R ; the choice is made by the environment. The sequential composition $P; Q$ behaves initially like P ; if it terminates successfully, then the control passes to Q .

Parallel composition, hiding and interruption are other CSP operators used in the model for a state-based use case. The process $P \parallel [X] Q$ stands for the generalised parallel composition of the processes P and Q with synchronisation set X . This expression states that P and Q must synchronise on events that belong to X . Each process can progress independently for events that are not in X . This composition terminates successfully if, and only if, the left- and

the right-hand side processes do terminate. Likewise, consider the process $P \setminus X$ that communicates all its events, except the events that belong to X , which become internal (invisible): $\setminus$ stands for the hiding operator. We also introduce the process $P \triangle Q$, which indicates that Q can interrupt the behaviour of P if an event offered by Q is communicated.

Moreover, consider the replicated external choice of CSP $\square x : A \bullet F(x)$, where x is a value from the set A , and $F(x)$ is any process expression involving x . This construction behaves as the process $F(a_1) \square \dots \square F(a_k)$, for $A = \{a_1, \dots, a_k\}$. Additionally, consider the process $RUN(s) = \square ev : s \bullet ev \rightarrow RUN(s)$ that continuously offers the events from the set s .

CSP Traces Model. Our test generation approach is based on refinement verification using the traces model of CSP [17]. The traces of a process P , given by $\mathcal{T}(P)$, correspond to the set of all possible sequences of events P can communicate. For instance, the traces model for the *STOP* process is the set $\{\langle \rangle\}$; the traces of the process $a \rightarrow P$ is $\{\langle \rangle\} \cup \{\langle a \rangle \hat{\ } t \mid t \in \mathcal{T}(P)\}$; and the traces of an external choice $P \square Q$ is $\mathcal{T}(P) \cup \mathcal{T}(Q)$. A complete definition for all CSP operators can be found in [17].

It is possible to compare the traces semantics of two processes by refinement verification using one of the CSP refinement checking tools, such as FDR [10]. A process Q refines a process P in the traces model, say $P \sqsubseteq_{\tau} Q$, if, and only if, $\mathcal{T}(Q) \subseteq \mathcal{T}(P)$. If the refinement does not hold, FDR yields a trace (the shortest counter-example), say ce , such that $ce \in \mathcal{T}(Q)$ but $ce \notin \mathcal{T}(P)$. For instance, $(P \square Q) RTP$ holds, since $\mathcal{T}(P)$ is a subset of $\mathcal{T}(P \square Q)$. However, the relation $STOP RT_{accept} \rightarrow STOP$ does not, since $\langle accept \rangle \in \mathcal{T}(accept \rightarrow STOP)$ but $\langle accept \rangle \notin \mathcal{T}(STOP)$. Thus, the trace $\langle accept \rangle$ is a counter-example for the above refinement expression.

Test Generation with CSP Test Purposes. Test scenarios are traces from the specification that describe particular behaviours to be tested. A test case is a CSP process that is constructed from the trace, as detailed in [15]. The mapping of the CSP events of a test case into the respective CNL elements produces a test case suite that can be executed directly by a tester. In our approach, test scenarios are yielded as counter-examples of refinement verifications using CSP test purposes [14], which are CSP process representing a partial specification of the test scenarios to be selected from a specification. We summarise how CSP test purposes are used to generate test scenarios.

Let the processes *ACCEPT*, *ANY* and *UNTIL* be primitive test purposes used in the construction of CSP test purposes. The process $ACCEPT = accept \rightarrow STOP$ is used to mark test scenarios by communicating the mark event *accept*, which does not belong to the alphabet of the specification process. The primitive $ANY(evset, next) = \square ev : evset \bullet ev \rightarrow next$ performs basic selection. It selects the events offered by the specification that belong to *evset*. If any of these events is communicated, it behaves as *next*. Otherwise, it deadlocks. The process $UNTIL(alpha, evset, next) = RUN(alpha \setminus evset) \triangle ANY(evset, next)$,

where α is the alphabet of the specification, selects all sequences offered by the specification events until it engages in some event that belongs to $evset$.

The CSP sample test purpose $TP1 = UNTIL(\alpha_S, \{event\}, ACCEPT)$ selects test scenarios that reach the event $event$ in a specification process S , such that $event \in \alpha_S$. Consider the expression $PP1 = S \parallel [\alpha_S] TP1$ that represents the parallel product between S and $TP1$. The refinement expression $S \sqsubseteq_r PP1$ yields test scenarios as counter-examples, if there are test scenarios that match $TP1$. The right-hand side of the expression behaves as the process S while the event $event$ is not communicated. After the communication of $event$, the $accept$ event is communicated and the composition deadlocks. Consequently, the traces ending with $accept$ belong to $PP1$ but not to $\mathcal{T}(S)$. Moreover, the verification of this refinement using FDR does not hold and yields a test scenario as the shortest counter-example trace.

In general, FDR yields only the shortest counter-example trace, thus, for obtaining lengthier counter-examples it is necessary to run other verifications in an interactive manner. Further details on interactive test generation approach, and on case construction from test scenarios can be found in [15].

3 State-Based Use Case

The notation we propose for state-based use cases is introduced via a case study that describes a feature of a 4G mobile phone. The grammar for this notation is presented in [2]. The standards for mobile networking have evolved through several generations: 2nd generation (2G), 3rd generation (3G), and 4th generation (4G). A mobile phone has three configurations related to internet over the carrier network: (i) the network mode defines which standard generation is being used; (ii) the mobile data enables or disables the access to the internet over the carrier network; and (iii) the wireless (wifi) enables or disables internet access over a wireless router. The use cases described below introduce scenarios where these configurations conflict with each other.

First, we introduce new types (Table 1). The type OnOff contains two values: On and Off, which are useful for variables that represent toggle switches. The type NetworkModes contains all possible networking compatibility standards. The N_2G and N_3G values represent connection modes that operate exclusively in a 2G and 3G network, respectively. The N_3G_2G network mode works for both 3G and 2G networks, prioritizing the 3G connection, although still attempting to connect to a 2G network whenever the 3G connection fails. Analogously, the N_4G_3G mode works for both 4G and 3G network. Finally, the N_4G_3G_2G network mode targets a 4G connection, but should work with all previous generations in the described order. Our CNL also allows a new type to be introduced either by indexing or from a base type. Indexing associates a tag with the values defined in a range. For example, the construction Phone[1,3] creates a new type composed of the values Phone.1, Phone.2 and Phone.3. A new type can also be created by associating a tag to the elements of an existing type (in general allowing disjoint unions). For instance, the new type Car.OnOff

contains the values Car.On and Car.Off. Table 1 also shows the declaration of the constants Non_4G_Modes and All_4G_Modes. The constant Non_4G_Modes is a set of NetworkModes that does not operate under 4G, while All_4G_Modes is the set complement of Non_4G_Modes.

Table 1. Type, constant and variable declarations.

New Type

Id	Description	Value
OnOff	Type of a toggle switch	On, Off
NetworkModes	Types of network compatibilities	N_2G, N_3G, N_3G_2G, N_4G_3G, N_4G_3G_2G

Constants

Id	Description	Value
Non_4G_Modes	Set of non 4G modes	{N_2G, N_3G, N_3G_2G}
All_4G_Modes	Set of 4G modes	{N_4G_3G, N_4G_3G_2G}

Variables

Id	Description	VarType	Value
selectedMode	Network mode configuration	NetworkModes	N_3G_2G
mobileData	Mobile data configuration	OnOff	On
wifi	Wifi configuration	OnOff	Off

Variables are also declared in Table 1. The variable selectedMode is of type NetworkModes and has an initial value of N_3G_2G. This variable stores the configuration of the network mode of the mobile phone. The variables mobileData and wifi are of type OnOff. When mobileData is On, the mobile phone can access the internet over the carrier network, and, when the wifi is On, the mobile phone can access the internet over a wireless router. The initial value of the mobileData is On and the wifi is Off.

Table 2 shows, at the top, the header of the Use Case 01 (UC01). It declares the use case title (Network Mode vs Mobile Data) and states that it includes the Use Case 02 (UC02) at steps 3M and 4M. We explain how the include works when introducing the UC02 in the sequel. The main flow specification of UC01 is also shown in Table 2. A step is defined by an identifier (Step ID), the user action (Action), the system state before the action is carried out (System State), and the expected result (System Response). In Step 1M, the user disables the mobile data. Regardless of the current system state, the device turns off the mobile data. Internally, the variable mobileData is assigned to Off. The notation between two occurrences of the symbol % is the subset of the CNL that manipulates state changes, inputs and outputs. In Step 2M the user selects one of the 4G network modes. The CNL sentence contained in the Action cell **%Input x : NetworkModes from All_4G_Modes%** takes an input from the user and assigns it to the local variable x. The variable x is of type NetworkModes restricted to the subset All_4G_Modes. As the mobile data was disabled in Step 1M, the system does not allow this selection to be done. The sentence (**%Output x%**) produces the value to replace the \$ sign. Therefore, the message “\$ is not available while mobile data is off or wifi is on.” is displayed, where the \$ sign

is replaced by the value of x . Step 3M enables the mobile data. No change is made on the network mode (so, it remains N_3G_2G, the default value defined in Table 1), but the variable `mobileData` is now On. Finally, Step 4M tries again to set up a new network mode. This time, the system successfully assigns the input x to the variable `selectedMode`. Table 3 shows the main flow of UC02. This use case describes the scenario where the mobile phone makes a call, sends an SMS (Short Message Service) and an MMS (Multimedia Messaging Service). A call and the sending of an SMS do not require the internet to be turned on; but an MMS does. The main flow assumes that the System State has either the `mobileData` or the `wifi` (or both) enabled. In this case, the SMS and the MMS are sent. Also, the call is made.

Table 2. UC01 main flow.

UC01 - Network Mode vs Mobile Data
Include

Use Case Id	Position
UC02	3M, 4M

Main Flow

Step Id	Action	System State	System Response
1M	Disable the mobile data.		The device disables the mobile data. %mobileData := Off%
2M	Manually select a network mode containing the 4G option. %Input x : NetworkModes from All_4G_Modes%		The warning “\$ is not available while mobile data is off or wifi is on.” is displayed and the network mode remains unchanged. %Output x%
3M	Enable the mobile data.		The mobile data is enabled. The selected network remains \$. %mobileData := On, Output selectedMode%
4M	Manually select a network mode containing the 4G option. %Input x : NetworkModes from All_4G_Modes%		The device selects the new network mode : \$ %selectedMode := x, Output x%

The alternative flow is also described in Table 3. The control flow jumps from the main flow START point, i.e. before step 1A happens, and returns to the main flow END point, i.e. after 1A finishes. The alternative flow introduces the scenario where no internet access is available, i.e. the System State has both the `mobileData` and the `wifi` off. As a result, the call is made and the SMS is sent, but the MMS is not delivered.

The UC02 is included as part of the UC01. Table 2 shows an include declaration at steps 3M and 4M. This means that the UC02 runs twice. Firstly, it runs before step 3M and, once completed, it returns to UC01 to run step 3M. In the second time, it runs before 4M followed by the execution of 4M. So, the UC01 executes the following steps: 1M, 2M, UC02, 3M, UC02, and 4M.

Table 3. UC02 main and alternative flow.

UC02 - Wifi and Mobile Data

Main Flow

Step Id	Action	System State	System Response
1M	Make a call, and send an SMS and an MMS.	Either the wifi or the mobile data is turned on. %(mobileData==On) or (wifi==On)%	The device makes the call and sends both the SMS and the MMS.

Alternative Flow

From: START

To: END

Step Id	Action	System State	System Response
1A	Make a call, and send an SMS and an MMS.	Both the wifi and the mobile data are turned off. %(mobileData==Off) and (wifi==Off)%	The device makes the call and sends the SMS, but is not able to send the MMS.

4 Automatic Generation of CSP Test Models

This section describes our approach for generating CSP test models from use cases. The process which yields a CSP model is split in three steps. First, the syntax of the CNL is checked; second, an XML representing the use cases is generated; and finally, the CSP test model that represents the use case document is yielded. We further detail these steps in what follows.

Validating Use Case Documents. Our use cases are described as part of a fix tabular template where its structure is well formed by construction. So part of the syntactical structure of the use cases does not need to be checked. However, we still need to verify the well-formedness of the CNL embedded in the template. So we built a parser and a type checker to carry out syntactic and semantic analysis and point out errors to the user. If no errors are reported, the process continues and we move on to the second step.

Building an XML Representation. A use case is translated into an intermediate format as an XML file because it does not dependent on any formal notation. This simplifies future developments by giving the flexibility to use alternative formalisms for test generation. Figure 2 illustrates a portion of the XML schema for defining new types. The type OnOff is described in Fig. 2 as an enumeration. Note that each field in Table 1 becomes a tag in XML.

Generating the CSP Model. The final step yields the CSP test model, where each use case is modelled as a CSP process. Accordingly, every use case flow is also modelled as a process. If the use cases declare variables, then an additional CSP process that works as a RAM memory is created. In what follows we present the translation of some elements of the example introduced in Sect. 3.

The types, the constants and the variables declared in Table 1 are translated into CSP as shown in Fig. 3.

The variables are mapped into a datatype *Var* (line 01). New types are mapped into the datatype *Type* (line 02), which is the disjoint union of all types. Each type is declared by a tag that identifies the type followed by the


```

<newType>
  <id>OnOff</id>
  <description>Type of a toggle switch</description>
  <newTypeElements>
    <enumeration>
      <element>On</element>
      <element>Off</element>
    </enumeration>
  </newTypeElements>
</newType>

```

Fig. 2. The XML for the new type OnOff.

01	<i>datatype</i> <i>Var</i> = <i>wifi</i> <i>mobileData</i> <i>selectedMode</i>
02	<i>datatype</i> <i>Type</i> = <i>t1.t_OnOff</i> <i>t2.t_NetworkModes</i>
03	
04	<i>datatype</i> <i>t_OnOff</i> = <i>On</i> <i>Off</i>
05	<i>datatype</i> <i>t_NetworkModes</i> = <i>N_2G</i> <i>N_3G</i> <i>N_3G_2G</i> <i>N_4G_3G</i>
06	
07	<i>Non_4G_Modes</i> = {(<i>N_2G</i>), (<i>N_3G</i>), (<i>N_3G_2G</i>)}
08	<i>All_4G_Modes</i> = {(<i>N_4G_3G</i>), (<i>N_4G_3G_2G</i>)}
09	
10	<i>INIT</i> = {(<i>wifi</i> , <i>t1.Off</i>), (<i>mobileData</i> , <i>t1.On</i>), (<i>selectedMode</i> , <i>t2.N_3G_2G</i>)}
11	
12	<i>channel</i> <i>get</i> , <i>set</i> : <i>Var.Type</i>

Fig. 3. CSP model for data definitions.

separator “.” and the respective set of values (line 02). The values for the types are mapped into CSP datatypes (lines 04 and 05). For instance, *t1.On* and *t1.Off* are instances of the type *t1*. Each constant is mapped to a constant in CSP (lines 07 and 08). The initial binding for the variables (Table 1) is defined as a set of pairs (variable name, initial value) named *INIT* (line 10). The channels *get* and *set* (line 12) communicate variable names and values. These channels are used to read the value of a variable from the memory and to update a value of a variable to the memory, respectively. For instance, the event *set.mobileData.t1.Off* stores *Off* into the variable *mobileData*. Figure 4 shows the translation of the UC01.

The control flow is defined by the process *F1_FLOW* in Fig. 4 (line 3), which behaves as the process *UC1_1M*. The process *UC1_1M* models the step 1M of the use case UC01 (compare this process with the step 1M shown in Table 2). The event *disableData* represents the user action and the event *dataDisabled* is the system response. The event *set!mobileData!t1!Off* updates the variable *mobileData* to the value *Off*. The event *mem_update* is a flag that indicates that a memory update has happened (this is used by the test purposes described in Sect. 5). The process *UC1_1M* behaves as *SKIP* followed by the behaviour of process *UC1_2M*, which models the behaviour for the step 2M. The process

01	<i>channel input, showNotAvailable, showMode, modeChanged : t_NetworkModes</i>
02	
03	<i>F1_FLOW = UC1_1M</i>
04	
05	<i>UC1_1M = disableData → dataDisabled → set!mobileData!t1!Off →</i>
06	<i>mem_update → SKIP; UC1_2M</i>
07	<i>UC1_2M = selectMode → input?x : {e e ∈ 4G_Modes} →</i>
08	<i>showNotAvailable!x → SKIP; UC1_3M</i>
09	<i>UC1_3M = UC2; get!selectedMode.t2?selectedMode → enableData →</i>
10	<i>showMode!selectedMode → set!mobileData!t1!On →</i>
11	<i>mem_update → SKIP; UC1_4M</i>
12	<i>UC1_4M = UC2; selectMode → input?x : {e e ∈ 4G_Modes} →</i>
13	<i>modeChanged!x → set!selectedMode!t2!x → mem_update → SKIP</i>

Fig. 4. CSP model for use case UC01.

UC1_2M uses the channels *input* and *showNotAvailable* (lines 07 and 08) to specify the values given as input, and the output value produced by the system. The notation $?x : \{e \mid e \in 4G_Modes\}$ in *input?x* denotes the value to be communicated is a choice of the environment. The process *UC1_3M* initially behaves as the process *UC2* that specifies the behaviour of the use case UC01 (due to inclusion). Then, it reads from the memory the current value for the variable *selectedMode* (*get!selectedMode!t2?selectedMode*), which is communicated by the channel *showMode* as the system response (line 10). The process *UC1_4M* starts at UC01 because of the use case inclusion. Then it takes as input one of the 4G modes and outputs it through the channel *modeChanged*. The memory for *selectedMode* is updated and the process terminates successfully (*SKIP*). We omit the model for UC02 due to space limitations.

The top-level process for the feature described in Sect. 3 is specified by the process *F1*, as follows.

$$\begin{aligned}
 F1 = & ((F1_FLOW; END1) \\
 & ||| \alpha_{F1_MEMORY} \cup \{success\} ||| \\
 & (F1_MEMORY \triangle END2)) \setminus (\alpha_{F1_MEMORY} \cup \{success\})
 \end{aligned}$$

This process is the parallel composition of the control flow *F1_FLOW* and the respective memory process *F1_MEMORY* synchronising on the alphabet of the memory plus the control event *success*. The process *F1_MEMORY* provides two services: the current value of a variable can be retrieved via the *get* event; and a new value of a variable can be stored via the *set* event. Furthermore, for each event in the form *get.var.type.val*, there is a copy *get'.var.type.val* event, which is used for the purpose of test selection (see Sect. 5). The auxiliary process *END1 = willsucc → success → SKIP* is used to flag successful termination via the *willsucc* event whenever a use case in *F1_FLOW* terminates. This process allows the successful termination of the process *F1_FLOW* by synchronising with the event *success* that is offered by the auxiliary process *END2 = success →*

SKIP, which interrupts the memory process whenever a use case in *F1_FLOW* terminates. Finally, the control events (α_{F1_MEMORY} and *success*) are hidden.

5 State-Based Test Purposes

We present the notation for authoring state-based test purposes. This language enables the partial specification of test scenarios based on the use case steps and states of the use case variables. New constructs have been added into the language to specify the selection of steps. State conditions are specified using boolean expressions. Each construct of the language is automatically translated to a CSP process that represents a test purpose, which is used for test generation in the way presented in Sect. 2. Thus, CSP notation is again hidden from the test designer. Table 4 presents the constructs for the test purposes notation (column Test Purpose) and the CSP processes that specify the constructs (column CSP Test Purpose).

Table 4. CSP model for test purposes.

	Test purpose	CSP test purpose
01	<i>next stepId</i> [, <i>TP</i>]	$ANY(\{act\}, UNTIL(\alpha_S, \{res\}, T(TP)))$
02	<i>not stepId</i> [, <i>TP</i>]	$ANY(A_I \setminus \{act\}, T(TP))$
03	<i>stepId</i> [, <i>TP</i>]	$UNTIL(\alpha_S, \{res\}, T(TP))$
04	<i>guard</i> [, <i>TP</i>]	$GUARD = get'.var1?var1 \rightarrow \dots \rightarrow get'.varN?varN \rightarrow$ $if(eval(guard)) then T(TP)$ $else UNTIL(\alpha_S, \{mem_update\}, GUARD)$
05	<i>STOP</i>	$UNTIL(\alpha_S, \{mem_update\},$ $get'.var1?var1 \rightarrow \dots \rightarrow get'.varN?varN \rightarrow$ $ACCEPT)$
06	<i>SUCCESS</i>	$willsucc \rightarrow T(STOP)$
07	<i>TP1 OR TP2</i>	$T(TP1) \sqcap T(TP2)$
08	<i>TP1 AND TP2</i>	$T(TP1) [\alpha_S \cup \{accept\}] T(TP2)$

Table 4 line 1 introduces the *next* construct that specifies test scenarios initiating with the step *stepId*. The notation *f#uc#step* is used to define *stepId*. This notation represents the full identification for a step whose id is *step*, and belongs to the use case *uc* of the feature *f*. For instance, *F1#UC01#2M* represents the step 2M of the use case UC1 that belongs to the feature F1 (in Sect. 3). Let the set α_S stand for the alphabet of the specification, and *act* and *res* CSP events that represent the action and the system response of the step *stepId*, respectively. The CSP model for this construct is defined in terms of the primitive test purposes *ANY* and *UNTIL* introduced in Sect. 2. After matching *act*, the test purpose accepts any event until it finds *res*. This guarantees that

the events between *act* and *res* are in the test scenario. In Table 4, $T(TP)$ stand for the CSP model for a test purpose TP. What the test purpose selects after the event *res* is (optionally) defined by a test purpose TP. If TP (preceded by a comma) is absent, it is implicitly assumed that TP is STOP (Table 4, line 5).

A test purpose in the form *not stepId*, TP (line 2 in Table 4) selects test scenarios that initiate with any step, except the step *stepId*. This constructor is complementary to the previous one. The CSP model for this construct selects any event in the input alphabet but *act*. The subsequent events are specified by TP. A step *stepId* not preceded by any construct (line 3), specifies test scenarios that reach the system response of *stepId*, which can be preceded by any other steps. The steps subsequent to step *stepId* are specified by TP.

Line 4 in Table 4 shows a state-based test purpose of the form *guard*, TP , where *guard* is a boolean expression that refers to the feature variables. The notation for describing such an expression is the same introduced in Sect. 3 for describing system conditions. Test scenarios in which *guard* is evaluated to true are the ones to be selected. The CSP model for *guard*, TP initially reads the current values of the variables using *get'* events, then evaluates the guard. If the guard holds, the selection continues according to TP; otherwise, the test purpose looks for a memory update event (to guarantee that the state changed) and the process recurses.

The test purpose STOP (Table 4, line 5) reads all variable values and includes a mark event. Its CSP model looks for a memory update event; after that it reads the variables in order to record their values at the end of the test scenarios. Finally, it behaves as *ACCEPT* (introduced in Sect. 2).

The test purpose *SUCCESS* (line 6) selects test scenarios that terminate successfully: those for which the CSP model communicates a $\checkmark$ event. The CSP model synchronises on the event *willsucc* that flags that the use case will terminate, then it behaves as $T(STOP)$.

The last two constructs in Table 4 (lines 7 and 8) enable test purposes to be combined. Let TP1 and TP2 be test purposes. The expression TP1 OR TP2 specifies the set of test scenarios that match TP1 plus the ones that match TP2. The CSP model is the external choice of the model for TP1 with the model for TP2. Remember that the parallel product (see Sect. 2) is the parallel composition of a test purpose TP with the specification S . According to the laws of CSP, we have that $S \parallel [\dots] T(TP1) \square T(TP2)$ equals $S \parallel [\dots] T(TP1) \square S \parallel [\dots] T(TP2)$, whose semantics is the union of $\mathcal{T}(S \parallel [\dots] T(TP1))$ with $\mathcal{T}(S \parallel [\dots] T(TP2))$. The expression TP1 AND TP2 specifies test scenarios that are common to TP1 and TP2. The CSP model for this expression is the parallel composition $T(TP1) \parallel [\dots] T(TP2)$, whose semantics is $\mathcal{T}(T(TP1)) \cap \mathcal{T}(T(TP2))$. Consequently, this composition selects traces that are common to both test purposes.

As an example, consider the CNL test purpose $TP2 = (\text{mobileData} == \text{Off}), F1\#UC1\#2M$ that selects test scenarios for which the value of the variable *mobileData* equals Off and that terminates in the step $F1\#UC1\#2M$, such that TP2 is the test purpose identifier (an abbreviation for *mobileData* == Off, ...). Thus, the CSP model for this test purpose is characterised by the processes that follow.

$$TP2 = \text{get'.mobileData?}v \rightarrow \text{if}(v == t1.\text{Off}) \text{ then } TP3 \\ \text{else } UNTIL(\alpha_{F1}, \{\text{mem_update}\}, TP2)$$

$$TP3 = UNTIL(\alpha_{F1}, \{ | \text{showNotAvailable} | \}, T(STOP))$$

$$T(STOP) = UNTIL(\alpha_{F1}, \{\text{mem_update}\}, \text{get'.wifi?}v_1 \rightarrow \\ \text{get'.mobileData?}v_2 \rightarrow \text{get'.selectedMode?}v_3 \rightarrow \text{ACCEPT})$$

Let $PP2 = F1 \parallel [\alpha_{F1}] TP2$ be the parallel product between $F1$ and $TP2$. The shortest test scenario yielded by the verification of the refinement expression $F1 \setminus \{\text{mem_update}\} \sqsubseteq_r PP2 \setminus \{\text{mem_update}\}$ is presented in the sequel, namely ts . The hiding of the control event mem_update removes such an event from the yielded test scenarios.

$$ts = \langle \text{get'.mobileData.t1.On, disableData, dataDisable,} \\ \text{get'.mobileData.t1.Off, selecteMode, input.N_4G_3G,} \\ \text{showNotAvailable.N_4G_3G, get'.wifi.t1.Off,} \\ \text{get'.mobileData.t1.Off, get'.selectedMode.t2.N_3G_2G, accept} \rangle$$

As expected, the get' events in the end of ts show that mobileData equals Off after the system output in $F1\#\text{UC01}\#2\text{M}$ (event $\text{showNotAvailable.N_4G_3G}$). The events in ts are translated to CNL counterparts with the reverse mapping from CNL to CSP used in Sect. 4 for generating the CSP model. This yields a CNL test case that is used for manual execution.

6 Tool Support

This section presents an overview of the TaRGeT tool with the necessary extensions to mechanise the entire strategy described in this paper, and, particularly, it gives a short demonstration of how the authoring of state-based use cases and test purposes is supported by the tool.

TaRGeT is developed using Java [16] and Eclipse RCP [13] with the idea to aggregate different capabilities in separate implementation units called plugins. A new plugin was added to support the new GUI interfaces for authoring state-based use cases and CNL test purpose. It also comprises a compiler that validates the CNL notation and translates from CNL to CSP, and a test generator component which calls FDR to generate test cases.

Data Definition Editor. Figure 5 shows the interface of the data definition editor, created to assist the test designer in the task of authoring data in our use case template. Several verifications are performed during the edition as syntactic and semantic checks to report errors.

Test Purpose Generation. TaRGeT has also been extended to consider state-based test purposes. Figure 6 illustrates the interface which allows the definition of test purposes. Each test purpose is given a unique identifier. This allows the user to create new test purposes composed of previously defined ones.

Data Definition Editor

▼ New Type

ID*	Description	Value*
OnOff	Type of a toggle switch	On, Off
NetworkModes	Types of network compatibilities	N_2G, N_3G, N_3G_2G, N_4G_3G, N_4G_3G_2G

▼ Constant

ID*	Description	Value*
Non_4G_Modes	Set of non 4G modes	{N_2G, N_3G, N_3G_2G}
All_4G_Modes	Set of 4G modes	{N_4G_3G, N_4G_3G_2G}

▼ Variable

ID*	Description	Type*	Value*
selectedMode	Indicates the currently selected...	NetworkModes	N_3G_2G
mobileData		OnOff	On
wifi		OnOff	Off

Fig. 5. The data definition editor.

Test Purpose Creation
Create your test purposes

Steps

- 1 - Mobile LTE Network
 - UC_01 - Normal Behavior
 - 1M - Disable the mobile data.

Current Test Purpose

Clean Ok

Created Test Purposes

Clean All Test Purposes

Generate Test Cases

Fig. 6. Test purpose creation

After the user finishes editing test purposes, the extended version of TaRGeT converts use cases to XML, which is translated to CSP as detailed in Sect. 4. Test purposes, after syntactic and semantic checks, are converted directly to CSP processes that are used for generating test scenarios as explained in Sect. 5. Finally, test scenarios are translated to test cases for test execution.

7 Conclusion

Use cases are widely used in industry in consolidated approaches to requirements specification. A use case specification typically describes, in the form of an event flow, in natural language, interactions between the user and the system. Unfortunately, natural language is hard to analyse and to process. In this work, we have proposed an approach to specify use cases in a Controlled Natural Language. Our use cases are able to specify both control flow and data aspects. The control flow is captured via the tabular structure of our use cases in addition to commands that take the flow to alternative, inclusion and extension of use cases. The data flow is embedded in the use case specification via the CNL for manipulating inputs, outputs and state variables.

Both control and data constructs are translated into a CSP model. By adopting CSP we can take advantage of tools like FDR to mechanise test generation, which is carried out as a process refinement computation: the counter-example produced by FDR is used to build a test case. In order to select and control which test cases should be generated, we have also shown how the CNL can be used to specify test purposes, which are also translated into CSP; once it runs in parallel with the CSP translation of the use cases, it automatically selects which test cases are produced. A tool fully integrated with FDR has been developed for use case editing and test case generation using test purposes.

Traditionally, automatic test case generation is carried out from formal languages that are typically not mastered by test designers; therefore this demands extra effort in the testing process: the test designer needs to translate the requirements to the formal language and the test cases produced back to another format (either natural language or a programming language). With a Controlled Natural Language embedded on a widely used requirements artifact, such as use cases, we conceal the formal aspects of test case generation.

Several works have proposed approaches to generate formal specifications from requirements to solve the problem of ambiguous, imprecise and unclear specifications [4, 8, 20–22]. However, differently from our approach, the formal notations used are not a process algebra, their formal models are not used with the purpose of test derivation, and the input, output and state are not analysed and processed as we do. For instance, Schwitter and Fuchs [20] propose a Controlled Natural Language that resembles a formal specification in logic and that can be translated to Prolog [24]. The notions of input, output and state that may appear in the CNL are translated as Prolog facts and not processed as part of a state based requirements. Similarly, Sinha *et al.* [21] analyse use cases for the purpose of edit-time, instantaneous inspection of their style and content. Inputs, outputs, and states are not processed as part of a system, but as part of a text to be checked for completeness, structure, flow, dependencies, etc. Some other works introduce approaches for test case generation from requirements described in a variety of notations [3, 5–7, 23]. Requirements are translated to a model that is used as the input for test generation. For instance, Carvalho *et al.* [7] introduce an approach for the generation of test vectors from a Controlled Natural Language that captures temporal requirements for the domain of controlling systems. Transition relations are used as the formal specification language. Somé and Cheng [23] take as input use cases written in natural language with a well defined syntax. Similarly to our work, their approach allows the use of step flows, condition and operations within the test cases. However, there is no notion of inputs and outputs and it is only possible to use data types provided by default. New types cannot be created by the user as in our approach. Another related approach is [6] that generates test cases for control system based on a natural language that supports timed reactive system. None of these approaches allow the specification of the test purposes in a Controlled Natural Language.

As future work, we intend to investigate how to produce executable test cases, particularly for the UIAutomator framework [1]. We also plan to explore the use of compression techniques of FDR and analyse efficiency gains in refinement verification. Concerning scalability, as a future work we plan to apply our strategy to more complex systems.

References

1. Developers, A.: UIAutomator (2015). <https://developer.android.com/tools/testing-support-library/index.html>
2. Bezerra, R.: Extração Automática de Modelos CSP a partir de Casos de Uso. Master’s thesis, Center of Informatics of Federal University of Pernambuco (2011)

3. Boddu, R., et al.: RETNA: from requirements to testing in a natural way. In: Proceedings of the RE 2004, pp. 262–271. IEEE, Washington (2004)
4. Brottier, E., Baudry, B., Traon, Y.L., Touzet, D., Nicolas, B.: Producing a global requirement model from multiple requirement specifications. In: Proceedings of the 11st edn. OC, EDOC 2007, p. 390. IEEE, Washington (2007)
5. Nebut, C., et al.: Automatic test generation: a use case driven approach. IEEE Trans. Softw. Eng. **32**(3), 140–155 (2006)
6. Carvalho, G., et al.: Test case generation from natural language requirements based on SCR specifications. In: Proceedings of the 28th Annual ACM Symposium on Applied Computing, SAC 2013, pp. 1217–1222. ACM, New York (2013)
7. Carvalho, G., Lapschies, F., Schulze, U., Peleska, J., Barros, Flávia: Model-based testing from controlled natural language requirements. In: Artho, C., Ölveczky, P.C. (eds.) FTSCS 2013. CCIS, vol. 419, pp. 19–35. Springer, Heidelberg (2014)
8. Drazan, J., Mencl, V.: Improved processing of textual use cases: deriving behavior specifications. In: van Leeuwen, J., Italiano, G.F., van der Hoek, W., Meinel, C., Sack, H., Plášil, F. (eds.) SOFSEM 2007. LNCS, vol. 4362, pp. 856–868. Springer, Heidelberg (2007)
9. Ferreira, F., Neves, L., Silva, M., Borba, P.: TaRGeT: a model based product line testing tool. In: Proceedings of CBSoft 2010 – Tools Panel (2010)
10. Gibson-Robinson, T., Armstrong, P., Boulgakov, A., Roscoe, A.W.: FDR3 17 a modern refinement checker for CSP. In: Tools and Algorithms for the Construction and Analysis of Systems, pp. 187–201 (2014)
11. Hoare, C.A.R.: Communicating Sequential Processes. Prentice Hall, Upper Saddle River (1985)
12. Jard, C., Jéron, T.: TGV: theory, principles and algorithms: a tool for the automatic synthesis of conformance test cases for non-deterministic reactive systems. Int. J. Softw. Technol. Transf. **7**(4), 297–315 (2005)
13. McAffer, J., et al.: Eclipse Rich Client Platform: Designing, Coding, and Packaging Java(TM) Applications. Addison-Wesley Professional, Lebanon (2005)
14. Nogueira, S., Sampaio, A., Mota, A.M.: Guided test generation from CSP models. In: Fitzgerald, J.S., Haxthausen, A.E., Yenigun, H. (eds.) ICTAC 2008. LNCS, vol. 5160, pp. 258–273. Springer, Heidelberg (2008)
15. Nogueira, S., Sampaio, A., Mota, A.: Test generation from state based use case models. Formal Aspects Comput. **26**(3), 441–490 (2014)
16. Oracle: Java JSE, July 2015. <http://www.oracle.com/>
17. Roscoe, A.W.: The Theory and Practice of Concurrency. Prentice Hall PTR, Upper Saddle River (1998)
18. Roscoe, A.W.: Understanding Concurrent Systems. Texts in Computer Science. Springer, London (2010)
19. Júnior, V.A.D.S., Vijaykumar, N.L.: Generating model-based test cases from natural language requirements for space application software. Software Qual. J. **20**(1), 77–143 (2012)
20. Schwitter, R., Fuchs, N.E.: Attempto - from specifications in controlled natural language towards executable specifications. In: CoRR cmp-lg/9603004 (1996)
21. Sinha, A., Sutton, M.S., Paradkar, A.: Text2test: automated inspection of natural language use cases. In: Proceedings of the ICST 2010, ICST 2010, pp. 155–164. IEEE Computer Society, Washington (2010)
22. Somé, S.S.: Supporting use case based requirements engineering. Inf. Soft. Technol. **48**(1), 43–58 (2006)

23. Somé, S.S., Cheng, X.: An approach for supporting system-level test scenarios generation from textual use cases. In: Proceedings of SAC 2008, pp. 724–729. ACM, New York (2008)
24. Sterling, L., Shapiro, E.: The Art of Prolog. MIT Press, Cambridge (1986)
25. Veanes, M., Campbell, C., Schulte, W., Tillmann, N.: Online testing with model programs. In: Proceedings of the 10th European Software Engineering Conference, ESEC/FSE-13, pp. 273–282. ACM, New York (2005)